

Home

Library

Profile

Stories

Stats

Following

Dev Genius

Python in Plain En...

Generative AI

Realworld AI Use C...

The Useful Tech

Level Up Coding

AI Mind

Deep concept

Top Python Librari...

The Mac Alchemist

More

Member-only story

Google Just Shipped 13 Agent Skills. I Plugged Them Into Gemini CLI and Watched Code Quality Jump.



Pawel

Follow

9 min read · Apr 29, 2026



395



3



5



One install command. Two prompts. The diff that proves Google quietly adopted Anthropic's open format as their official agent documentation layer.

I asked Gemini CLI the same question twice. Once with no skills installed. Once after running a single command — `gemini skills install https://github.com/google/skills.git`. The first answer used a deprecated SDK Google has been quietly retiring for a year. The second answer used the current unified SDK, with proper Pydantic schemas, environment-variable-driven Agent Platform configuration, and multi-turn tool execution. Same model. Same prompt. Different universe.

This is what Google shipped at Cloud Next 2026 on April 28: an official Agent Skills repository with 13 skills, distributed under Apache 2.0, working out of the box with Gemini CLI, Claude Code, Antigravity,

Codex, and any other agent that supports the Skills format. The format itself wasn't invented by Google — it was built and open-sourced by Anthropic. Google adopting it as their official documentation delivery layer is the actual story.

Google's own evaluations of the Gemini API skill report it pushes correct-API-code generation to **87% with Gemini 3 Flash and 96% with Gemini 3 Pro**. I wanted to see what those numbers look like in practice.

What Google actually shipped

The repository lives at github.com/google/skills. Thirteen skills out of the gate, organized into three categories.

Seven product skills: AlloyDB, BigQuery, Cloud Run, Cloud SQL, Firebase, the Gemini API on Agent Platform (formerly Vertex AI), and Google Kubernetes Engine.

Three Well-Architected Framework pillar skills: Security, Reliability, and Cost Optimization. These map directly to Google's WAF guidance and activate when the agent detects architectural decisions in flight.

Three recipe skills: Google Cloud Onboarding, Authentication, and Network Observability — operational playbooks rather than product documentation.

Installation in Gemini CLI is a single command:

```
gemini skills install https://github.com/google/skills.git
```

Verification:

```
gemini skills list
```

You should see all 13 skills listed as [Enabled] with their descriptions. Every skill landed in `~/.agents/skills/` — the format-agnostic path that Anthropic's spec defines and Gemini CLI now reads natively alongside its own `~/.gemini/skills/` directory.



Google's Official Agent Skills

github.com/google/skills · 13 skills · Launched April 2026 · Apache 2.0

PRODUCTS · 7

**alloydb-basics**
clusters, instances, backups; MCP integration

**BigQuery-basics**
Datasets, tables, jobs; BigQuery ML & Gemini

**Cloud Run-basics**
Services, jobs, worker pools (HTTP, scheduled)

**Cloud SQL-basics**
for MySQL, PostgreSQL, and SQL Server

**firebase-basics**
products for mobile & web apps (Firestore, Auth)

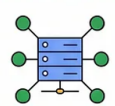
**gemini-api**
Gen AI SDK (Python, JS, Go) on Agent Platform

**GKE-basics**
Autopilot: networking, security, scaling, cost

WELL-ARCHITECTED PILLARS · 3

**google-cloud-waf-security**
Security guidance from the WAF (IAM, network)

**google-cloud-waf-cost-optimization**
Cost optimization guidance from the Cloud WAF

**google-cloud-waf-reliability**
Reliability guidance from the WAF (Resilience, HA)

RECIPES · 3

**google-cloud-recipe-auth**
Auth and ADC for users, services, secure access

**google-cloud-recipe-networking-observability**
VPC Flow Logs, NAT, firewall, latency diagnostics

**google-cloud-recipe-onboarding**
First steps: account, billing, projects, first deploy

Professional Agent Skills Visualization

Install: `gemini skills install https://github.com/google/skills.git` · format originally by Anthropic

Demo 1: Gemini API code generation

I asked Gemini CLI to generate production code for calling Gemini 3 Pro

https://medium.com/@meshuggah22/google-just-shipped-13-agent-skill...-them-into-gemini-cli-and-watched-code-quality-jump-988e54aacdb

Page 3 of 13

on Vertex AI / Agent Platform with structured JSON output (5-field schema), function calling for a `get_weather(city)` tool, and proper error handling.

Before — no skills installed

Concise. Wrong SDK. The code imports `vertexai` and the `vertexai.generative_models` module — the legacy SDK that Google has been actively deprecating in favor of `google-genai`. It targets `gemini-1.5-pro`, not the current 3.x family. Schemas are constructed manually with parameter dictionaries instead of Pydantic. The pattern would compile and run today, but it's the kind of code that would get flagged in any modern Google Cloud code review.

This is what's lurking in every model's training data: a year of deprecated examples that still appear correct because the SDK still works. There's no

way for the model to know it's pointing to the wrong import path without explicit guidance.

Skill activation

When I re-ran the same prompt with skills installed, Gemini CLI surfaced a consent prompt before doing anything.

This is progressive disclosure rendered as a UI moment. At session start, the agent only saw the `gemini-api` skill's name and description (about 80 tokens). When my prompt matched the description, the CLI asked permission to load the full `SKILL.md` plus the bundled `references/` directory — nine markdown files covering structured outputs and tools, embeddings, the Live API, media generation, model tuning, safety, and multimodal inputs. The agent gets exactly the context it needs, and only when it needs it.

I approved. The model then generated this:

After — with `gemini-api` skill activated

```
import os
import logging
from typing import List
from pydantic import BaseModel, Field
from google import genai
from google.genai import types

class AnalysisResult(BaseModel):
    """Schema for the structured output."""
    id: str = Field(..., description="Unique identifier for the analysis")
```

```
title: str = Field(..., description="A short, descriptive title")
summary: str = Field(..., description="A concise summary of the findings")
tags: List[str] = Field(..., description="A list of relevant tags or keywords")
score: float = Field(..., description="A confidence or importance score")

def get_weather(city: str) -> str:
    """Retrieves the current weather for a given city."""
    # ...

class GeminiProService:
    def __init__(self):
        # Picks up GOOGLE_CLOUD_PROJECT, GOOGLE_CLOUD_LOCATION,
        # GOOGLE_GENAI_USE_VERTEXAI=true from environment
        self.client = genai.Client()
        self.model_id = "gemini-3.1-pro-preview"

    def analyze_with_tools(self, user_prompt: str) -> AnalysisResult:
        config = types.GenerateContentConfig(
            tools=[get_weather],
            response_mime_type="application/json",
            response_json_schema=AnalysisResult,
        )
        response = self.client.models.generate_content(
            model=self.model_id,
            contents=user_prompt,
            config=config
        )
        # ... multi-turn tool handling via chat session
```

the diff is across every meaningful axis:

The skill didn't make the model smarter. It made the model correct. That distinction is what Google's 87% / 96% benchmark is measuring — not capability uplift, but accuracy uplift on a task where the model already had the capability and was just routing to the wrong vintage of documentation.

Demo 2: BigQuery cost optimization

I asked Gemini CLI for a cost optimization plan on a daily 2TB BigQuery aggregation that needed same-day freshness. Production workload, real budget pressure.

Before — no skills

Four bullets. Roughly 60 words. Mentions the right concepts (partitioning, clustering, materialized views, capacity-based billing) but at the level of a Stack Overflow answer. No numbers, no decision criteria, no execution order.

Skill activation

After — with bigquery-basics skill activated

To optimize a BigQuery production workload scanning 2TB daily (\$12.50/run on scans to a tiered architecture focusing on data pruning, pre-aggregation,

Phase 1: Storage & Pruning (Immediate Cost Reduction)
The goal is to stop scanning the full 2TB and only scan what has changed.

1. Time-Unit Partitioning:
 - * Action: Partition your events table by event_timestamp or a DATE column.
 - * Impact: If your query only needs "today's" data, partitioning reduces costs.
 - * Enforcement: Set require_partition_filter = true in the table settings.
2. Clustering:
 - * Action: Cluster by high-cardinality columns used in WHERE or GROUP BY clauses.
 - * Impact: BigQuery sorts data blocks by these columns. A query for a specific date can skip entire blocks, reducing costs by another 50-80%.
3. Storage Billing Model:
 - * Action: Evaluate Physical Storage pricing if your data is highly compressed. For large, static datasets, Physical pricing can be significantly cheaper.

Phase 2: Logic & Freshness (Architecture Shift)
Moving from daily full-refresh queries to incremental updates.

1. Materialized Views (MVs):
 - * Strategy: Create an MV that performs the basic aggregation (SELECT ... FROM ...).
 - * Benefits: MVs refresh incrementally in the background. When you query the MV, you get fresh data without scanning the raw table.
 - * Automatic Rewriting: If a user queries the raw table with logic that can be satisfied by the MV, BigQuery automatically rewrites the query to use the MV, saving costs.
2. Incremental MERGE Pattern:
 - * If MVs are too restrictive, use a Scheduled Query or Dataform to refresh an MV incrementally.

```
1      MERGE `summary_table` T
2      USING (SELECT ... FROM `events` WHERE event_timestamp > last_refresh_time) S
3      ON T.id = S.id
4      WHEN MATCHED THEN UPDATE ...
5      WHEN NOT MATCHED THEN INSERT ...
```

3. BigQuery Continuous Queries:
 - * For same-day freshness (near real-time), use Continuous Queries. They allow you to query a table without waiting for a daily batch.

Phase 3: Capacity Management (Financial Predictability)
Once the workload is optimized, switch from On-Demand to Capacity pricing.

1. BigQuery Editions (Standard vs. Enterprise):
 - * Standard Edition: Good for predictable SQL workloads.
 - * Enterprise Edition: Required if you need advanced features like Columnar Storage or Bigtable without waiting for a daily batch.
2. Slot Reservations:
 - * At 2TB/day, you are scanning ~60TB/month (\$375).
 - * The "break-even" point for a 100-slot reservation (Standard Edition) is around 100TB/month.
 - * Recommendation: Stay on On-Demand after optimizing partitioning/clustering. Once your workload exceeds 400TB/month, switch to Autoscaling Slots.

Summary Execution Plan

Priority	Task	Tool
High	Partition by Date + Cluster by ID	bq update --requi
High	Create Materialized View for aggregation	SQL CREATE MATERI
Med	Switch to Physical Storage Billing	BigQuery Console
Med	Implement Incremental MERGE	Dataform or Sched
Low	Move to Slot Reservations	BigQuery Reservat

Pro Tip: Use dry_run in the BigQuery UI or CLI (bq query --dry_run) after to your target range (e.g., 50GB) before you actually execute.

Same model. Same prompt. The skill turned a competent generalist into a Google FinOps architect.

Why this works: progressive disclosure as architecture

I had all 13 skills installed during both demos. Only the relevant skill activated for each prompt — gemini-api for the code generation, bigquery-basics for the cost optimization. The other 11 skills cost nothing in tokens because their full bodies never loaded. Only the name and description sit in the agent's context at session start (under a hundred tokens per skill), enough for the model to recognize a match and request activation.

This is the architectural shift that makes the format work at scale. You don't choose between expertise and a clean context window — the format gives you both. A team can maintain dozens of skills covering security reviews, migration playbooks, data pipelines, framework conventions; the agent stays lean until expertise is needed; the right skill loads on demand and stays out of the way otherwise.

MCP solved access — how does an agent reach external systems. Skills solve delivery — how does an agent receive specialized knowledge. They're complementary. The combination starts to look like a real answer to the context-bloat problem that has been quietly breaking

production agents for the past year.

The format standardization story

The Skills format wasn't built by Google. It was built by Anthropic, open-sourced as a spec, and adopted incrementally by anyone who needed a portable way to package agent expertise. HuggingFace shipped their own skills repo first. Google just made it official with thirteen first-party skills covering their Cloud product surface.

Three labs. One format. That's not a coincidence — that's how an emerging standard becomes infrastructure. The skill I installed from github.com/google/skills works without modification in Claude Code, Codex, Cursor, Antigravity, and Gemini CLI. Same SKILL.md. Same metadata. Same activation semantics. The agent runtime is increasingly interchangeable; the knowledge layer is increasingly portable.

The next ninety days will tell whether AWS and Microsoft follow with official skills repos for their own cloud surfaces. If they do — and the gravity is pulling that direction — Skills become the de facto delivery format for agent-readable product documentation across every major cloud.

What this means if you ship agents on Google Cloud

If you're building anything against Google Cloud APIs and you have a coding agent in your loop, install this today. The 87% / 96% accuracy numbers Google published aren't marketing copy — they're the difference between code that compiles and code that compiles against the right SDK. The cost is one command. The benefit shows up in the next prompt.

If you're building cross-platform agents, the more interesting move is to start authoring your own skills in the same format. The skill that helps Gemini CLI ship good code on your stack will, without modification, help Claude Code and Codex do the same.

The repo:

```
gemini skills install https://github.com/google/skills.git
gemini skills list
```

Run it. Ask your CLI a question that would normally produce mediocre code. Watch which skill activates, which references load, which 11 stay quiet. That moment — the consent prompt with the file tree showing exactly what context is about to enter your session — is the architectural beat worth understanding. Everything else is execution.

If you found this useful, follow for more hands-on coverage of agentic AI infrastructure.

AI

Genai

Generative AI Tools

AI Agent

Google Cloud Platform



Written by Pawel

530 followers · 4 following

Follow

Gen AI director with a focus on AI & Data Strategy, FinOps, MLOps & LLM/LMMOps.

